

9_Monitors

Stack Canaries

- To overwrite the return address you need to overwrite the input buffer and keep overwriting whatever is on the stack until you hit the return address which you overwrite to whatever you want to return to
- One kind of countermeasure to this is to put a special value just above the return address in stack which also has to be overwritten to overwrite the return address
- Then we add some instructions to our function which check whether this value has been modified
- On function entry a random number is created and put next to the return address so that it gets overwritten if a buffer overflow attack happens
- If the number is modified crash the program to prevent giving control to the attacker

(not part of canaries but another method to prevent code injection via buffer overflow)

- Fixed sized inputs to prevent code injection on the stack

Operating system security

Access Control

- System makes a decision to grant or reject an access request

Trusted computing base

- Part of the system which we have to trust to use it
- OS does not trust apps - access control is enforced

Security Enforcement



- Several applications include access control
 - Databases, window servers, web servers, browsers, ...
- Some programming systems include access control to system resources
 - Java, Safe-Tcl, Ruby, Python, Perl – Jif, Flow Caml (information flow);
- Some systems recognize that programs may contribute to access control
 - User-level policy server for SELinux
 - Information Flow Control
- *Requirement: Ensure that all layers are using their authority in a manner consistent with system security goals*

Protection System

- Manages the authorization policy for a system (describes what operations each subject can perform on each object)
- Consists of protection state and protection state operations (state ops)

Mandatory Protection System



- Is a *protection system* that can be modified only by *trusted administration* that consists of
 - A *mandatory protection state* where the protection state is defined in terms of an immutable set of *labels* and the *operations that subject labels can perform on object labels*
 - A *labeling state* that assigns system subjects and objects to those labels in the mandatory protection state
 - A *transition state* that determines the legal ways that subjects and objects may be relabeled
- MPS is *immutable* to user-space process

(aka Mandatory access control)

- Immutable rules apply subjects to labels (in rows) and objects to labels (in columns)

Transition state

- Rules that decide the conditions on which subjects and objects change labels
- For example, you want to lower the permission of a subject accessing possibly adversary controlled resources

Challenges with managing MPS

- You need a central admin
- Current method: use **dynamic analysis** to set up mandatory access control (MAC) policies - run the program and collect the permissions used (functional policy)

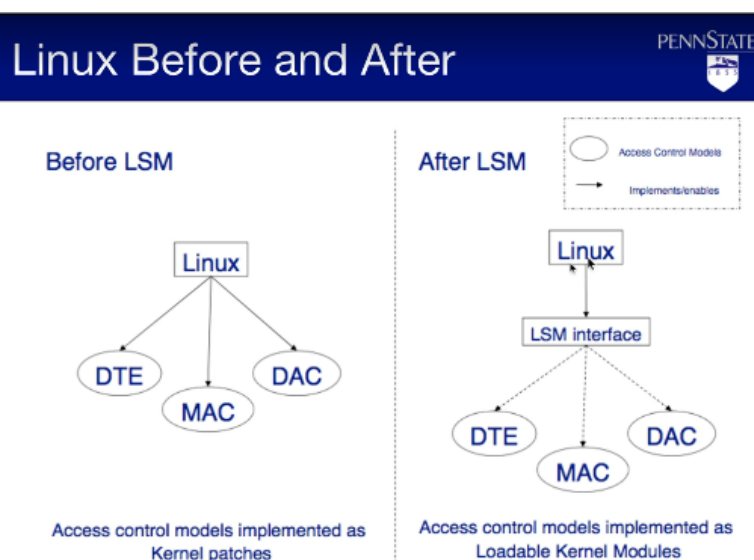
Reference monitor

- Is the one that does the enforcement if security goals
- Defines the goals of the MPS
- Components
 - Reference monitor interface (ex LSM - linux security modules)
 - Reference validation mechanism (ex SELinux)
 - Policy store (ex policy database)

- **Complete Mediation**
 - The reference validation mechanism must always be invoked
- **Tamperproof**
 - The reference validation mechanism must be tamperproof
- **Verifiable**
 - The reference validation mechanism must be subject to analysis and tests, the completeness of which must be assured

Linux security modules

- By default linux implements discretionary access control
 - Done using `chmod` to grant and remove privileges to files
- To implement mandatory access control you need to use linux security modules



- Requirements of LSM
 - Generic interface

- Conceptually simple
- Efficient
- Minimally invasive
- Tamperproof
 - Ensured by only interacting with the modules through an API
 - You as a user can't simply load a security module, to load a security module it needs to be compiled into the kernel which means that the kernel developers need to approve it
- Complete mediation
 - To ensure this add a security hook which is a part of every system call (this is basically an authorization query to the runtime monitor)
 - Sensitive data types are extended with security fields

LSM Hooks

PENN

- Function calls that can be overridden by security modules to manage security fields and mediate access to Kernel objects.
- Hooks called via function pointers stored in `security->ops` table
- Hooks are primarily "restrictive"

LSM Security Fields

- Enable security modules to associate security information to Kernel objects
- Implemented as void* pointers
- Completely managed by security modules

- LSM act as a reference monitor

LSM Tasks



- Linux Kernel modified in 5 ways:
 - Opaque security fields added to certain kernel data structures
 - Security hook function calls inserted at various points with the kernel code
 - A generic security system call added
 - Function to allow modules to register and unregister as security modules
 - Move capabilities logic into an optional security module

SELinux

(security enhanced linux)

- Is a linux security module
- Defining a proper policy to ensure integrity constraints is very difficult, so the way SELinux does it is it provides a comprehensive control of access which aims to restrict root processes and follow the least privilege principle in general
- **SELinux policy rules**
 - **Protection state** - ALLOW specified operations between specified subject label and specified object label
 - **Labelling state** - when files are created, decide which label the file has (default is to label the object to the same as its creator)
 - **Transition state** - define when labels are supposed to be updated

- Example policy for the `passwd` command (command to change your password)
 - For user to run passwd program
 - › Only passwd should have permission to modify `/etc/shadow`
 - Need permission to execute the passwd program
 - › `allow user_t passwd_exec_t:file execute` (user can exec `/usr/bin/passwd`)
 - › `allow user_t passwd_t:process transition` (user gets passwd perms)
 - Must transition to `passwd_t` from `user_t`
 - › `allow passwd_t passwd_exec_t:file entrypoint` (run w/ passwd perms)
 - › `type_transition user_t passwd_exec_t:process passwd_t`
 - Passwd can the perform the operation
 - › `allow passwd_t shadow_t:file {read write}` (can edit passwd file)
- After defining your policies (above example is for a single command) then you still have to do the following to do mandatory access control
 - Make sure services are aware of SELinux
 - Get the policy installed in the kernel
 - Manage the policy